

P1

mulher(M).

homem(H). ^{progenitor}
^{pai/mãe}
progenitor(P, F). ^{filho/filha}

pai(A,B):- progenitor(A,B),
homem(A).

mae(A,B):- progenitor(A,B),
mulher(A).

irmao_generico(A,B):- mae(C,A), mae(C,B),
pai(D,A), pai(D,B),
A \= B.

^{irmão de B}
irma(A,B):- irmao_generico(A,B),
mulher(A).

^{avós de B}
avos(A,B):- progenitor(A,X),
progenitor(X,B).

^{tios de B}
tios(A,B):- irmao_generico(A,X),
progenitor(X,B).

primos(A,B):- tios(X,B),
progenitor(X,A).

antepassado(A,B):- progenitor(A,B).

antepassado(A,B):- progenitor(A,X),
antepassado(X,B).

descendente(A,B):- progenitor(B,A).

descendente(A,B):- progenitor(B,X),
descendente(X,A).

meio_irmao(A,B):- pai(P,A), pai(P,B),
mae(M1,A), mae(M2,B),
M1 \= M2, A \= B. ^{Não necessário}

meio_irmao(A,B):- pai(P1,A), pai(P2,B),
mae(M,A), mae(M,B),
P1 \= P2.

primos_segundo_grau(A,B):- avos(X,A), avos(Y,B),
irmao_generico(X,Y).

parente(A,B):- antepassado(X,A),
antepassado(X,B),
A \= B.

parente(A,B):- descendente(X,A),
descendente(X,B),
A \= B.

a(X,Y).

a(X,Y):- e(Y,X).

cam(A,B):- cam(A,B,A).

P2

num(z).

num(s(x)):- num(x).

num(z,0).

num(s(x),SY):- num(X,Y),
SY is Y+1.

le(^oz,-).

le(s(A),s(B)):- le(A,B).

lt(z,s(-)).

lt(s(A),s(B)):- lt(A,B).

soma(z,x,x):- num(x).

soma(s(x),Y,s(Z)):- soma(X,Y,Z).

sub(A,B,X):- soma(X,B,A).

mult(z,x,z):- num(x).

mult(s(A),B,X):- mult(A,B,Y),
soma(B,Y,X).

% div(z,A,x):- mult(x,A,z).

div(A,B,X):- mult(B,X,A).

div(A,B,Q,R):- mult(B,Q,X),
soma(X,R,A),
lt(R,B).

divisor_comum(A,B,C):- num(C),
div(A,C,X),
div(B,C,X).

multigo_comum(A,B,C):- num(C),
mult(A,-,C),
mult(B,-,C).

^{lt?}
min(A,B,A):- le(A,B).

^{lt?}
min(A,B,B):- le(B,A).

^{lt?}
max(A,B,A):- le(B,A).

^{lt?}
max(A,B,B):- le(A,B).

primo(A):- lt(B,A),
B = s(s(-)),
lt(C,A),
mult(C,B,A),
!,
fail.

primo(A):- num(A).

P3

```
catena([], L, L).

catena([X|Xs], L, [X|Y]) :- catena(Xs, L, Y).

membro(X, [_|_]).

membro(X, [_|L]) :- membro(X, L).

membro(X, Y) :- catena(, [X|], Y).

prefixo([], _).

prefixo([X|A], [X, B]) :- prefxo(A, B).

prefixo(X, Y) :- catena(X, -, Y).

sufixo(A, A).

sufixo(A, [_|B]) :- sufixo(A, B).

sufixo(X, Y) :- catena(, X, Y).

sublista(S, L) :- prefxo(P, L),
                  sufixo(S, P).

sublista(S, L) :- prefxo(S, L).

sublista(S, [_|L]) :- sublista(S, L).

% tamanho([], 0).

% tamanho([_|T], X) :- tamanho(T, Y),
%                  X is Y+1.

tamanho([], z).

tamanho([_|T], s(X)) :- tamanho(T, X).

inverte(L, R) :- inverte(L, [], R).

inverte([], R, R).

inverte([A|B], X, R) :- inverte(B, [A|X], R).

nrev([], []).

nrev([X|A], B) :- nrev(A, AR),
                  catena(AR, [X], B).
```

P4

```
num(z).

num(s(x)) :- num(x).

soma(z, x, x) :- num(x).

soma(s(x), Y, s(Z)) :- soma(X, Y, Z).

mult(z, x, z) :- num(x).

mult(s(A), B, X) :- mult(A, B, Y),
                    soma(B, Y, X).

le(z, _).

le(s(A), s(B)) :- le(A, B).

catena([], L, L).

catena([X|Xs], L, [X|Y]) :- catena(Xs, L, Y).

membro(X, [_|_]).

membro(X, [_|L]) :- membro(X, L).

membro(X, Y) :- catena(, [X|], Y).

prefixo([], _).

prefixo([X|A], [X, B]) :- prefxo(A, B).

prefixo(X, Y) :- catena(X, -, Y).

sufixo(A, A).

sufixo(A, [_|B]) :- sufixo(A, B).

sufixo(X, Y) :- catena(, X, Y).

sublista(S, L) :- prefxo(P, L),
                  sufixo(S, P).

sublista(S, L) :- prefxo(S, L).

sublista(S, [_|L]) :- sublista(S, L).

tamanho([], z).

tamanho([_|T], s(X)) :- tamanho(T, X).

sequencia(A, A, [A]).

sequencia(A, B, [A|X]) :- sequencia(s(A), B, X).

double([], []).

double([A|X], [A, A|B]) :- double(X, B).

↪adjacentes
adj(E1, E2, [E1, E2|_]).

adj(E1, E2, [E2, E1|_]).

adj(E1, E2, [_|L]) :- adj(E1, E2, L).

↪select
sel(E, [E|L], L).

sel(E, [X|L], [X|M]) :- sel(E, L, M).

somaLista(L, S) :- soma_(L, z, S).

soma_([], X, X).

soma_([A|B], X, S) :- somaLista(A, X, Y), soma_(B, Y, S).

produto([], x, x).

produto([A|B], x, P) :- mult(A, x, Y), produto(B, Y, P).

produtoLista(L, P) :- produto_(L, s(z), P).
```



```
ordenado?
ord(L3).

ord(L_J).

ord([X, Y|L3]) :- le(X, Y), ord(Y|L).

perm(L3, L3).

perm([X|L1], L2) :- sel(X, L2, R), perm(L1, R).

sort(L, S) :- perm(L, S), ord(S).
```

Cargica

```
cargica(S) :- atom_chars(S, Lista), %Converte átomo para lista de char
reverse(Lista, Lista). %Verifica se a lista é igual à sua inversa
```

```
PS
%menor divisor comum
mdc(I, 0, I).

mdc(I, J, MDC) :- J>0,
R is I mod J,
mdc(J, R, MDC).

mdc(N, N, N).

mdc(A, B, C) :- A>B,
X is A-B,
mdc(X, B, C).

mdc(A, B, C) :- A<B,
X is B-A,
mdc(A, X, C).

%factorial
fact(0, 1).

fact(N, F) :- N>0,
N1 is N-1,
fact(N1, F1),
F is N*F1.

%melhor
fact(N, F) :- fact(N, 1, F).

fact(0, F, F).

fact(N, T, F) :- N>0,
T1 is T*N,
N1 is N-1,
fact(N1, T1, F).

%melhor
somalista(L3, 0).

somalista([I|Is], S1) :- Somalista(Is, SI),
S is I+SI.

somalista(Is, S) :- somalista(Is, 0, S).

somalista([I|Is], S, S) :-
T1 is T+I,
somalista(Is, T1, S).

%Interpretador de autômato finito não-determinístico (NFA)
aceita(S) :- inicial(Q), aceita(Q, S).

aceita(F, L3) :- final(F).

aceita(S, Xs) :- trans(S, L3, NS), aceita(NS, Xs).

aceita(S, [x|Xs]) :- trans(S, x, NS), aceita(NS, Xs).

Lê standard input e escreve no standard output, passando as
maiúsculas para minúsculas.

copy(IN, OUT) :- see(IN), tell(OUT), copy, seen, told.

copy :- get0(C), copy(C)

copy(-1) :- !.

copy(C) :- minusc(C), M is C+32, put(M), copy. %Altera o caractêr
atual caso seja maiúscula.

copy(C) :- \+ minusc(C), put(C), copy. %Copia o caractêr sem alterações
caso já seja minúscula.

minusc(L) :- 0'a <= L,
L <= 0'z.

maiusc(L) :- 0'A <= L,
L <= 0'Z.

to_ln(IN, OUT) :- see(IN), tell(OUT), copy, seen, told.

to_ln :- copy.

working_directory(D) - unifica D com a diretoria corrente.

directory_files(D, Fs) - unifica Fs com a lista de ficheiros na
diretoria D.

file_property(F, PROP)- unifica PROP com uma propriedade
do ficheiro F; pode ser, nomeadamente,
absolute_file_name(X), real_file_name(X),
type(X), size(S).
```


'a → Qualquer tipo
(+) → int
(+). → float

P6

Par

let par = function x → x mod 2 = 0;;

let rec... rec apenas se utiliza em definições recursivas, onde a função ou variável pode se referir a si mesma.

Ímpar

let impar x = not (par x);;

Divisível por

let divisivel_por = function x → function y → (y mod x) = 0;;

let divisivel_por x y = (y mod x) = 0;;

1 + 2;; = (+) 1 2;;

Par (Divisível por)

let par = divisivel_por 2

Fatorial

let rec fact n = if n < 1 then 1 else n * fact (n-1);;

let rec fact = function

0 → 1

| x → x * fact (x-1);;

let nulo = function

| 0 → "nulo"

| _ → "não nulo";;

Match (Semelhante ao switch em C)

match expr with p1 → e1 | p2 → e2 | ... | pN → eN

Comprimento Lista

let rec length = function

[] → 0

| _ :: x → 1 + length x;;

Notas:

Carregar um ficheiro source:

#use "xpto.ml";;

Ter acesso a funções de sistema:

#load "unix.cma";;

Mudar diretoria corrente:

Unix.chdir : string → unit

Membro membro : 'a list → 'a → bool

let rec membro = function

[] → (function _ → false)

| x :: y → function z → if z = x then true
else membro y z;;

Conta conta : 'a → 'a list → int

Conta o número de ocorrências do primeiro argumento na lista.

let rec conta x = function

[] → 0

| y :: z → (if x = y then 1 else 0) + conta x z;;

Clone

Clona a lista original

let rec clone = function

[] → []

| a :: b → a :: clone b;;

Clown

Soma um valor x a toda a lista original.

let rec clown = function

[] → (function x → [x])

| a :: b → (function x → a + x :: clown b x);;

Append 'a list → 'a list → 'a list

Catena as duas listas dos parâmetros.

let rec append = function

[] → (function x → x)

| a :: b → (function x → a :: append b x);;

Soma_Lista int list → int

Devolve a soma dos elementos da lista.

let rec soma_lista = function

[] → 0

| a :: b → a + soma_lista b;;

Remove 'a → 'a list → 'a list

Remove a primeira ocorrência do elemento que é o primeiro argumento, da lista que é o segundo.

let rec remove x = function

[] → []

| a :: b → if x = a then b else remove x b;;

Remove_tudo

Remove todas as ocorrências.

let rec remove_tudo x = function

[] → []

| a :: b → (if x = a then b else a) :: remove_tudo x b;;

Contagem 'a list → ('a * int) list

Devolve uma lista com pares de cada elemento e o seu número de ocorrências.

let rec contagem = function

[] → []

| a :: b → (a₊₁, (conta a b_b + 1)) :: (contagem (remove_tudo a b));;

P7

→ Define valores do tipo "forma"

```
type forma = Quadrado of float
            | Circulo of float
            | Retangulo of (float*float)
            | Ponto
            | Segmento of (float*float);;
```

```
let area = function
  Circulo x → 3.14 [*.] x *. x
  | Retangulo (a,b) → a *. b
  | Quadrado y → y *. y
  | Ponto → 0.0
  | Segmento _ → 0.0 ;;
```

Numerais

```
type numeral = Z | S of numeral;;
```

While

```
while expr1 do expr2 done
```

For

```
for nome = expr1 to expr2 do expr3 done
for nome = expr1 to expr2 downto expr3 done
```

Estas expressões apenas fazem sentido se haver efeitos secundários (input/output ou afetação)

Ex: for a=1 to 10 do print-int a; print-string "\n"; done;;

outputs 1 2 3 4 5 ...

→ comprimir listas

```
let rec comp = fun f n → function [] → n | x::y → f x (comp f n y);;
```

Ex: comp (+) 0 [1; 2; 3];;

0 elemento neutro

Out -> int=6

Ex: comp (*) 1 [1; 2; 3; 4];;

Out -> int=24

Árvore binária de pesquisa

```
type 'a abp =
  None
  | No of ('a * 'a abp * 'a abp);; No (a.esq, valor, a.dir)
```

Looking 'a abp → 'a → bool

Devolve true se o segundo argumento estiver na árvore que é o primeiro argumento.

```
let rec lookup = function
  None → (function _ → false)
  | No (key, left, right) → (function z → if z=key then true
    else lookup (if z<key then left else right) z);;
```

Insert 'a abp → 'a → 'a abp

Insere o segundo argumento na árvore que é o primeiro.

```
let rec insert = function
  None → (function x → No(x, None, None))
  | (No(key, left, right) → (function x → if x=key then No(key, left, right)
    else if x<key then No(key, insert left x, right)
    else No(key, left, insert right x));;
```

Delete 'a abp → 'a → 'a abp

```
let rec replace_leftmost = function
  None → (function x → x)
  | No(key, left, right) → (function x →
    → No(key, (replace_leftmost left x), right));;
```

```
let rec Delete = function
  None → (function _ → None)
  | No(key, left, right) → (function x → if x<key then
    No(key, Delete left x, right)
    else if x>key then No(key, left, Delete right x)
    else if x=key
    replace_leftmost right left);;
```

Walk 'a abp → ('a → unit) → unit

Atravessa a árvore do primeiro argumento em ordem e chama a função que é o segundo argumento sobre cada valor da árvore que encontrar nesse percurso.

```
let rec walk : 'a abp → ('a → unit) → unit = function
  None → (function _ → ())
  | No(key, left, right) →
    (function f → walk left f;
     f key;
     walk right f);;
```

Programação com Conjuntos

findall - apresenta todas as soluções, porém pode conter repetições e não considera variáveis livres.

bagof - apresenta todas as soluções, porém pode conter repetições e considera variáveis livres.

setof - apresenta todas as soluções sem repetições, melhorado? por ordem e considera variáveis livres.